GUC
German University in Cairo

CSIS 402 : COMPUTER ORGANIZATION & SYSTEM PROGRAMMING

INTEL 80X86 ADDRESSING MODES

LECTURE 6

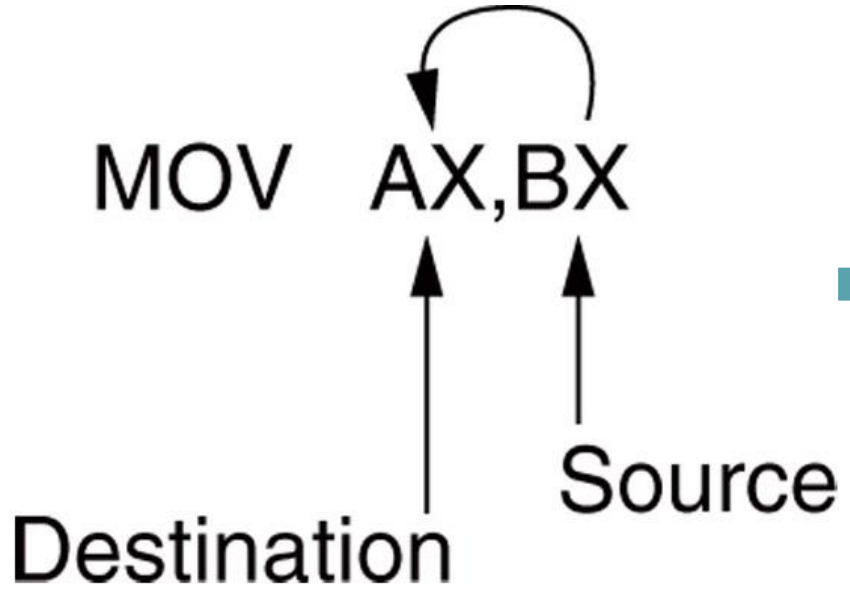
Dr. Sarah Azzam

Slides Based on Lecture Slides by Assoc. Prof. Dr. Amr Talaat

DATA ADDRESSING MODES

MOV INSTRUCTION PROVIDES A BASIS FOR EXPLANATION OF DATA-ADDRESSING MODES

opcode

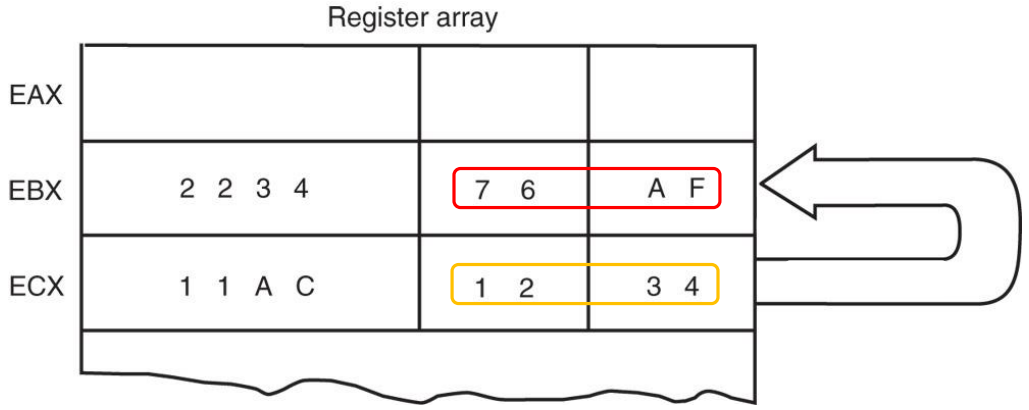


Transfers the word contents of *source register* **BX** to *destination registers* **AX**

an **opcode**, or operation code, tells the microprocessor which operation to perform

DATA ADDRESSING MODES

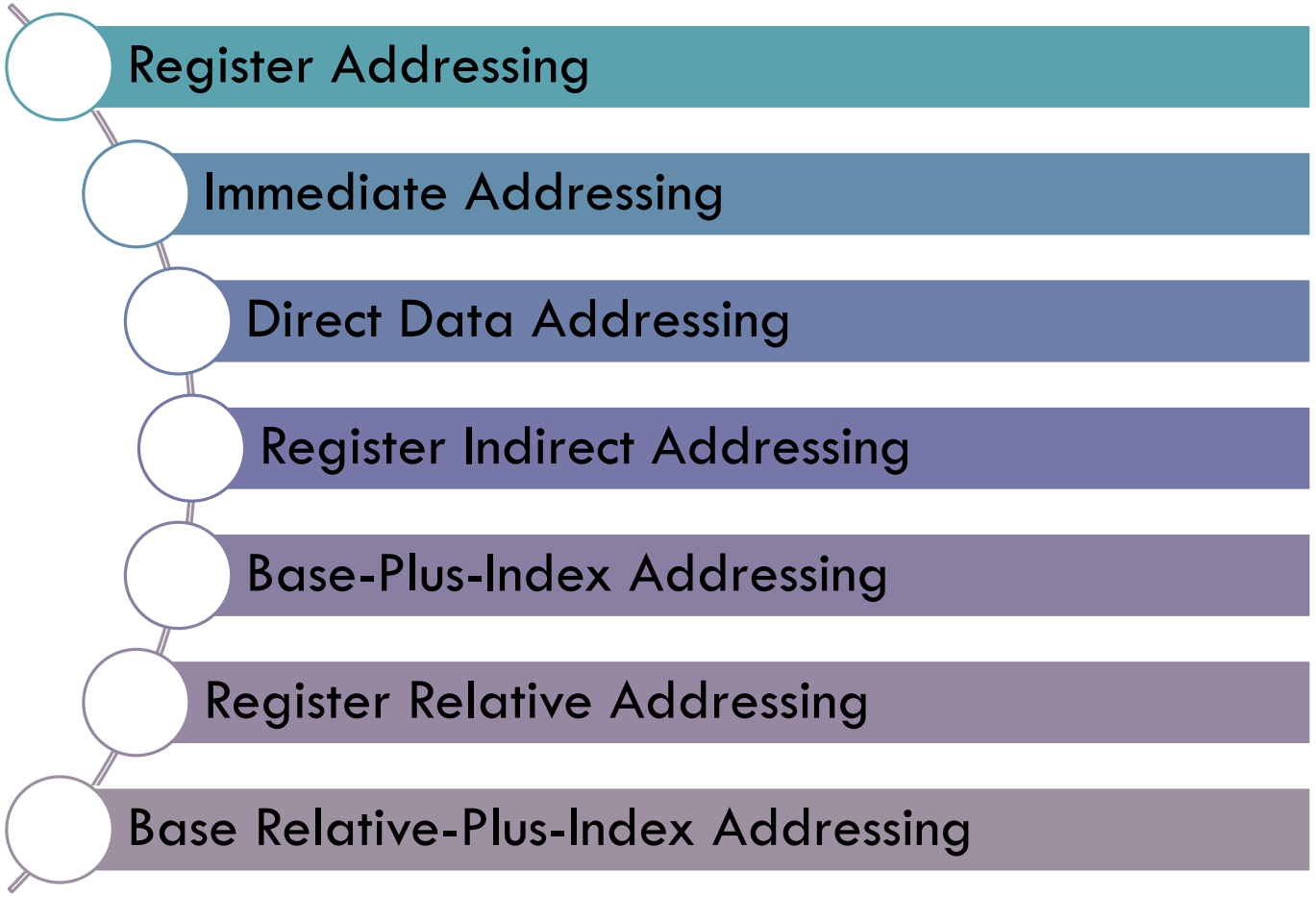
MOV BX, CX



- The source register's contents do not change.
- The destination register's contents do change.
- The contents of the destination register or destination memory location change for all instructions except the CMP and TEST instructions.
- Note that only the rightmost 16 bits of register EBX change. The MOV BX, CX instruction does not affect the leftmost 16 bits of register EBX.

DATA ADDRESSING MODES

The data addressing modes are as follows:



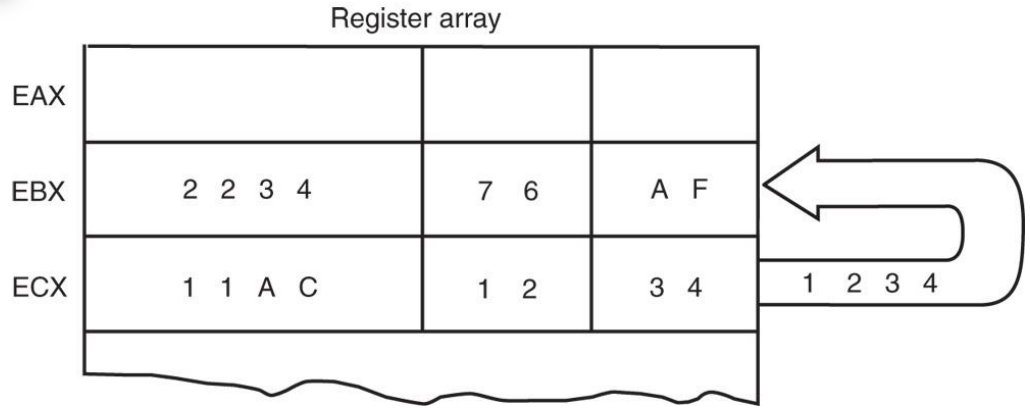
REGISTER ADDRESSING

		80286	80386 +	80486 +
Register	8-bit	16-bit	32-bit	64-bit
Accumulator	AH, AL	AX	EAX	RAX
Index	BH, BL	BX	EBX	RBX
Count	CH, CL	CX	ECX	RCX
Data	DH, DL	DX	EDX	RDX
Stack Pointer		SP	ESP	RSP
Base Pointer		BP	EBP	RBP
Src. Index		SI	ESI	RSI
Dest. Ind		DI	EDI	RDI
				R8 – R15

REGISTER ADDRESSING

EXAMPLE

MOV BX, CX



REGISTER ADDRESSING

- It is **important** for instructions to use **registers** that are **the same size**.
  - ***never*** mix an 8-bit \with a 16-bit register, an 8- or a 16-bit register with a 32-bit register
  - this is not allowed by the microprocessor and results in an error when assembled
- **None** of the MOV instructions cause **changes** in the **flag bits**. The flag bits are modified in case of arithmetic and logic operations.

REGISTER ADDRESSING



- A MOV instruction from **segment-to-segment** register is **not allowed**.
- It is **not allowed** to change the **CS register** using a **MOV instruction**.

REGISTER ADDRESSING

State whether the following instructions are allowed (valid) or not.

Explain what happens after execution of the allowed instructions.

Instruction	
MOV AL, BL	**(Allowed) Copies the content of BL to AL (8 bits)**
MOV CH, CL	**(Allowed) Copies the content of CL to CH (8 bits)**
MOV AX, CX	**(Allowed) Copies the content of CX to AX (16 bits)**
MOV SP, BP	**(Allowed) Copies the content of BP to SP (16 bits)**
MOV DS, CX	**(Allowed) Copies the content of CX to DS (16 bits)**
MOV ES, DS	**(Not Allowed) Segment-to-segment**
MOV BL, DX	**(Not Allowed) Register Sizes mismatch**
MOV CS, AX	**(Not Allowed) Code segment register should not be the destination register**

IMMEDIATE ADDRESSING

Term ***immediate*** implies that data immediately follow the hexadecimal opcode in the memory.

- immediate data are constant data
- data transferred from a register or memory location are variable data

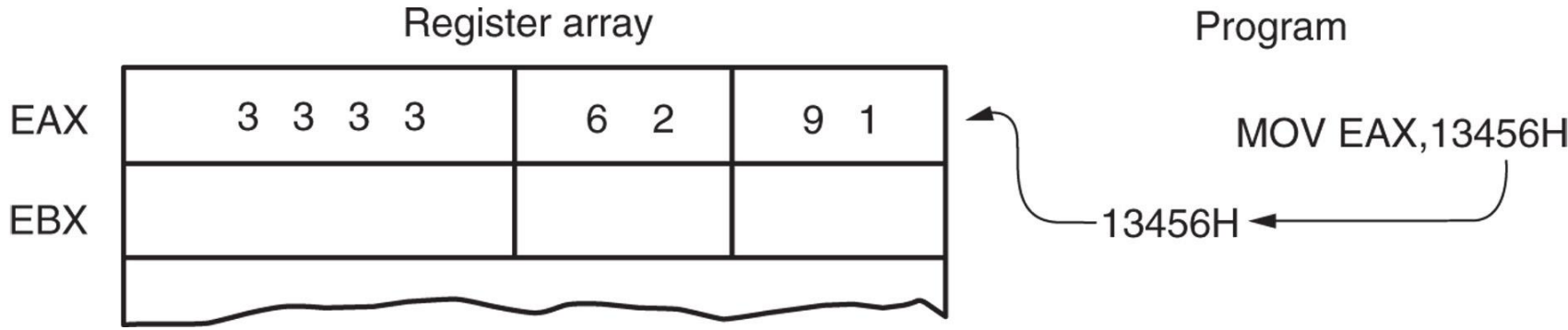
Immediate addressing operates upon a byte or word of data.

IMMEDIATE ADDRESSING

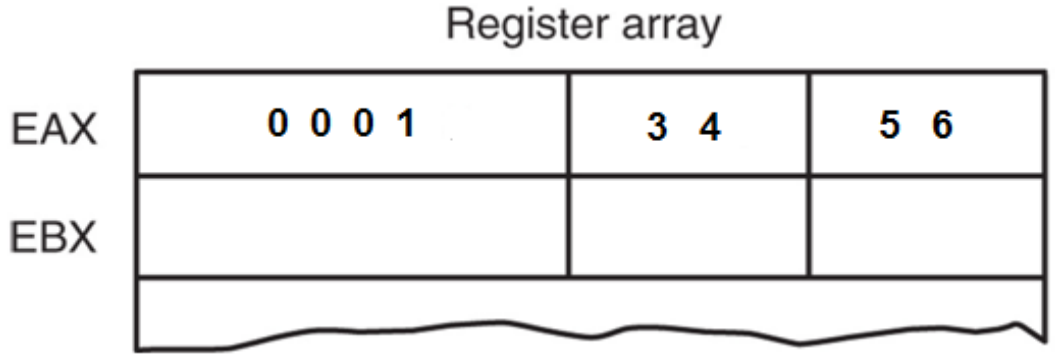
EXAMPLE

MOV EAX, 13456H

Before the instruction executed



After the instruction executed



- the source data overwrites the destination data.

IMMEDIATE ADDRESSING

- In symbolic assembly language, the symbol # precedes immediate data in some assemblers.
  - **MOV AX,#3456H** instruction is an example
- Most assemblers do not use the # symbol, but represent immediate data as in the **MOV AX,3456H** instruction.
  - an older assembler used with some Hewlett-Packard logic development does, as may others

IMMEDIATE ADDRESSING

- The symbolic assembler portrays immediate data in many ways.

The letter **H** appends hexadecimal data.

- If hexadecimal data begin with a letter, the assembler requires the data start with a **0**.
  - to represent a hexadecimal F2, **0F2H** is used in assembly language.
- Decimal data are represented as it is and require no special codes or adjustments.
  - an example is the 100 decimal in the **MOV AL,100** instruction
- An ASCII-coded character or characters may be depicted in the immediate form if the ASCII data are enclosed in apostrophes.
- Binary data are represented if the binary number is followed by the letter B or in some assemblers, the letter Y.

IMMEDIATE ADDRESSING

State whether the following instructions are allowed (valid) or not.

Explain what happens after execution of the allowed instructions.

Instruction	
MOV BL, 44	(Allowed) **Copies 44 decimal (2CH) into BL (8 bits)**
MOV AX, 44H	(Allowed) **Copies 004H into AX (16 bits)**
MOV SI, 0	(Allowed) **Copies 0000H into SI (16 bits)**
MOV CH, 100	(Allowed) **Copies 100 decimal (64H) into CH (8 bits)**
MOV CL, 11001110B	(Allowed) **Copies 11001110 binary into CL (8 bits)**
MOV EBX, 12340000H	(Allowed) **Copies 12340000H into EBX (32 bits)**
MOV EAX, 100B	(Allowed) **Copies 100 binary to EAX (32 bits)**
MOV RCX, 100H	(Allowed) **Copies 100H to RCX (64 bits)**

DIRECT DATA ADDRESSING

- Applied to many instructions in a typical program.
- Two basic forms of **Direct Data Addressing**:
  - ***Direct Addressing***, which applies to a **MOV** between a **memory location** and **AL, AX, or EAX**
  - ***Displacement Addressing***, which applies to almost any instruction in the instruction set
- **Address** is formed by adding the ***displacement*** to the **default data segment address** or an alternate segment address.

DIRECT ADDRESSING

Direct addressing with a **MOV** instruction **transfers data between a memory location, located within the data segment, and a register**.

MOV CX,DATA → loads CX from the data segment memory location DATA (1234H).

- DATA is a symbolic memory location, while 1234H is the actual hexadecimal location.

DIRECT DATA ADDRESSING

EXAMPLE

DS=1512H

```
MOV AL, 99H
MOV [3518], AL
```

After the instructions execution

Physical address18638 will have 99H

Why ?

Logical address DS:3518

DIRECT DATA ADDRESSING

EXAMPLE

MOV ES:[2000H], AL

After the instructions execution

AL register content is copied into extra Segment memory at offset address 2000H

REGISTER INDIRECT ADDRESSING

- Allows data to be addressed at **any memory location** through an **offset address** held in any of the following registers: **BP, BX, DI, and SI**.
- In addition, 80386 and above allow register indirect addressing with any extended register except ESP.

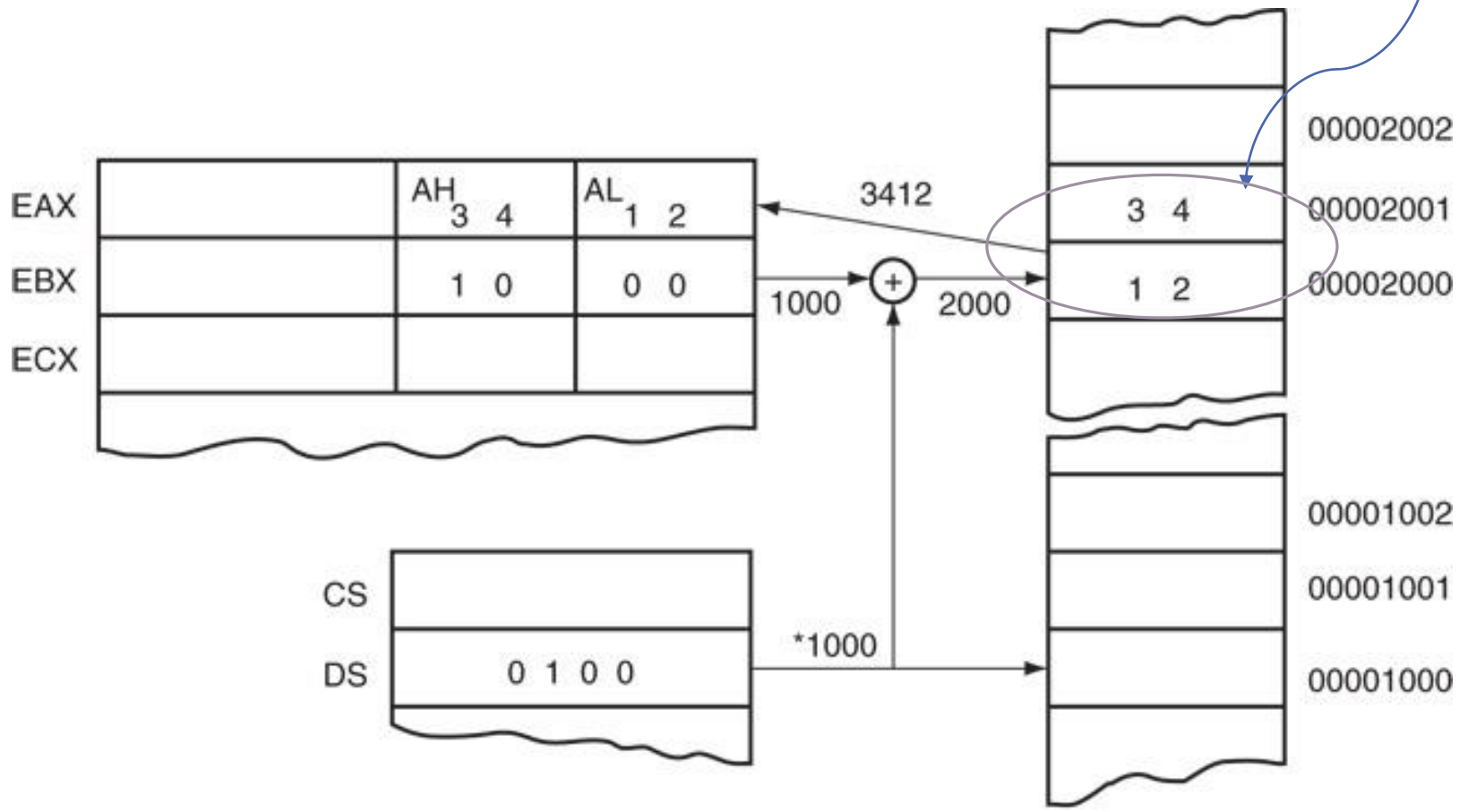
REGISTER INDIRECT ADDRESSING

Little Indian Convention

EXAMPLE

MOV AX, [BX]

BX=1000H, DS=0100H



*After DS is appended with a 0.

REGISTER INDIRECT ADDRESSING

- The **data segment** is used by default with **register indirect addressing** or any other mode that uses **BX, DI, or SI** to address memory.
- If the **BP** register addresses memory, the **stack segment** is used by default.
  - these settings are considered the default for these four index and base registers
- For the 80386 and above,
  - **EBP** addresses memory in the **stack segment** by default.
  - **EAX, EBX, ECX, EDX, EDI, and ESI** address memory in the **data segment** by default.

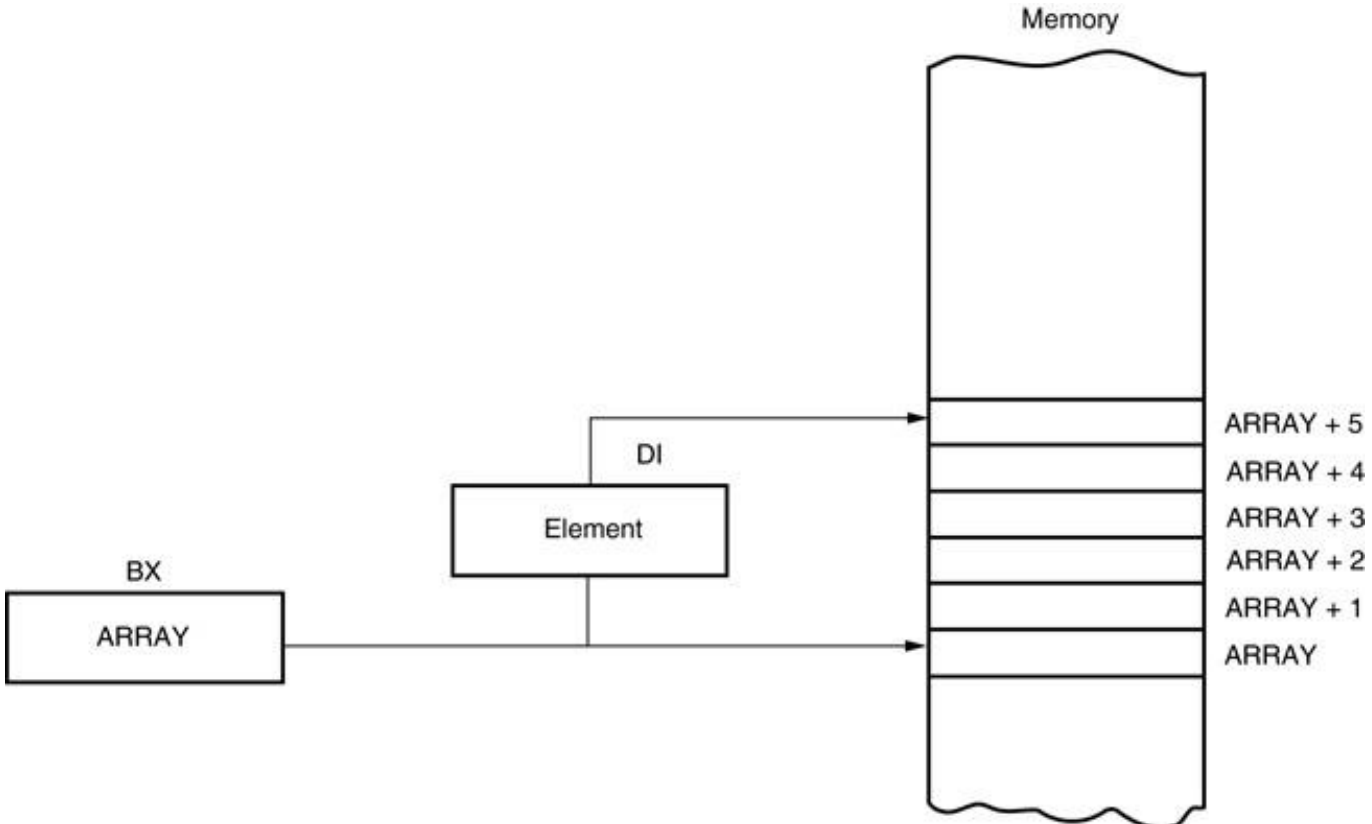
BASE-PLUS-INDEX ADDRESSING

- Similar to **indirect addressing** because it **indirectly addresses memory data**.
- The **base register** often holds the **beginning location of a memory array**.
  - the **index register** holds the **relative position of an element in the array**
  - whenever BP addresses memory data, both the **stack segment** register and BP generate the effective address

BASE-PLUS-INDEX ADDRESSING

An example of the Base-Plus-Index addressing modes.

Here an element (DI) of an array (BX) is addressed.



BASE-PLUS-INDEX ADDRESSING

Locating Data with Base-Plus-Index Addressing

MOV DX, [BX+DI]

- The Intel assembler requires this addressing mode appear as [BX][DI] instead of [BX + DI].
- The MOV DX,[BX + DI] instruction is MOV DX,[BX][DI] for a program written for the Intel ASM assembler.

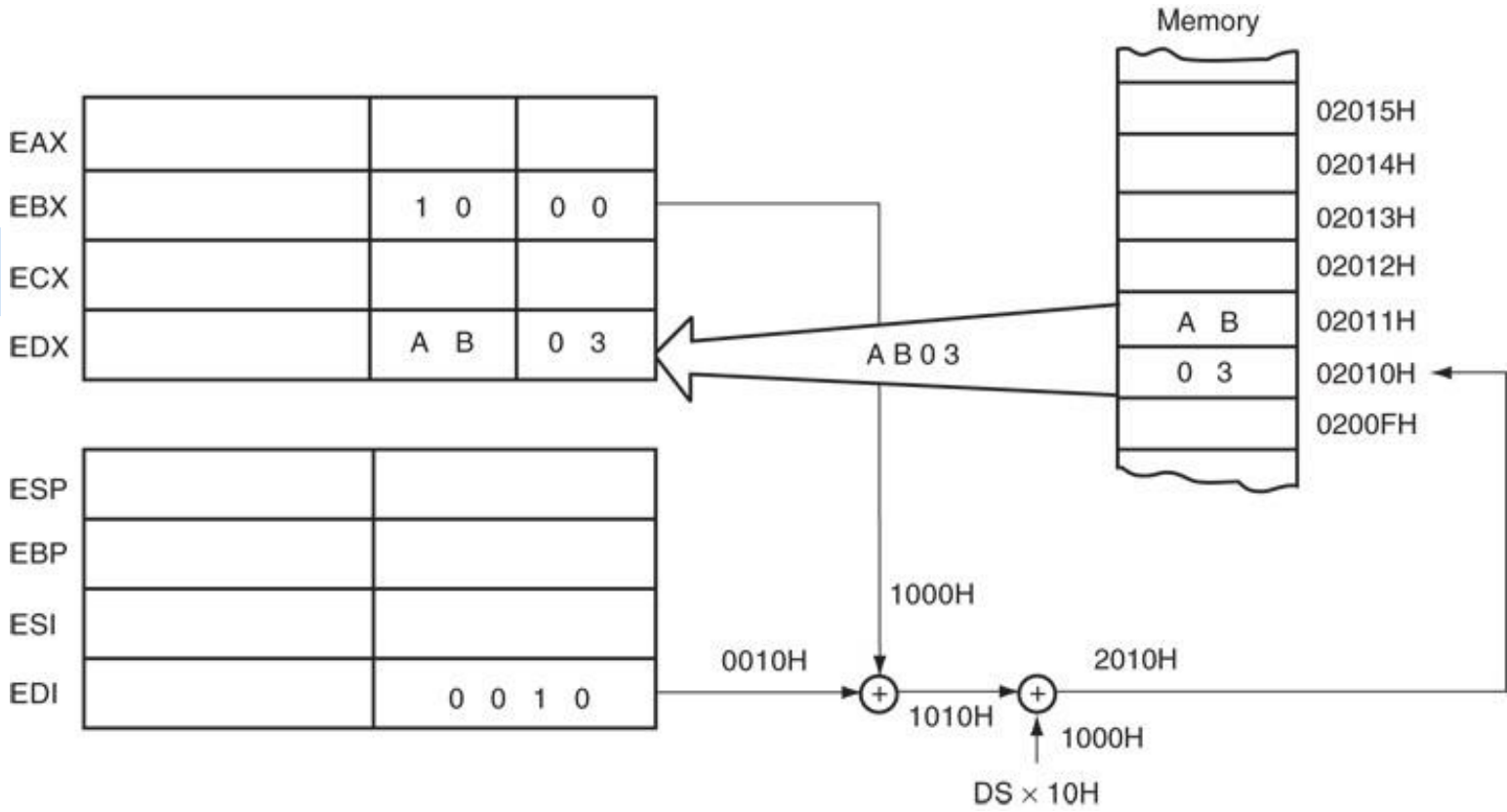
BASE-PLUS-INDEX ADDRESSING

EXAMPLE

MOV DX, [BX+DI]

BX=1000H, DS=0100H, DI=0010H

Little Indian Convention



REGISTER RELATIVE ADDRESSING

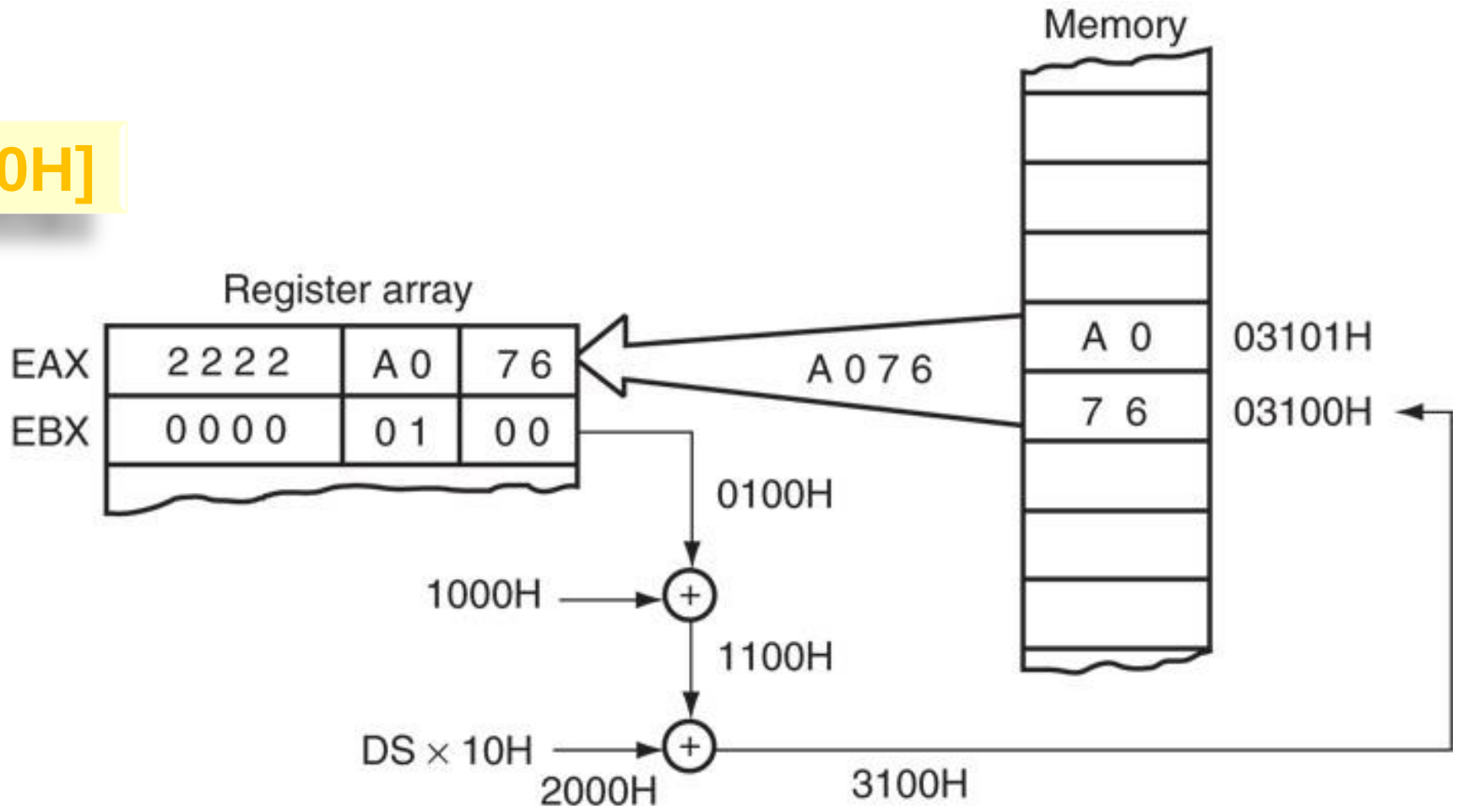
- Similar to **base-plus-index addressing** and **displacement addressing.**
  - **data in a segment of memory** are addressed by adding the displacement to the contents of a base or an index register **(BP, BX, DI, or SI)**.

REGISTER RELATIVE ADDRESSING

EXAMPLE

MOV AX, [BX+1000H]

BX=0100H, DS=0200H



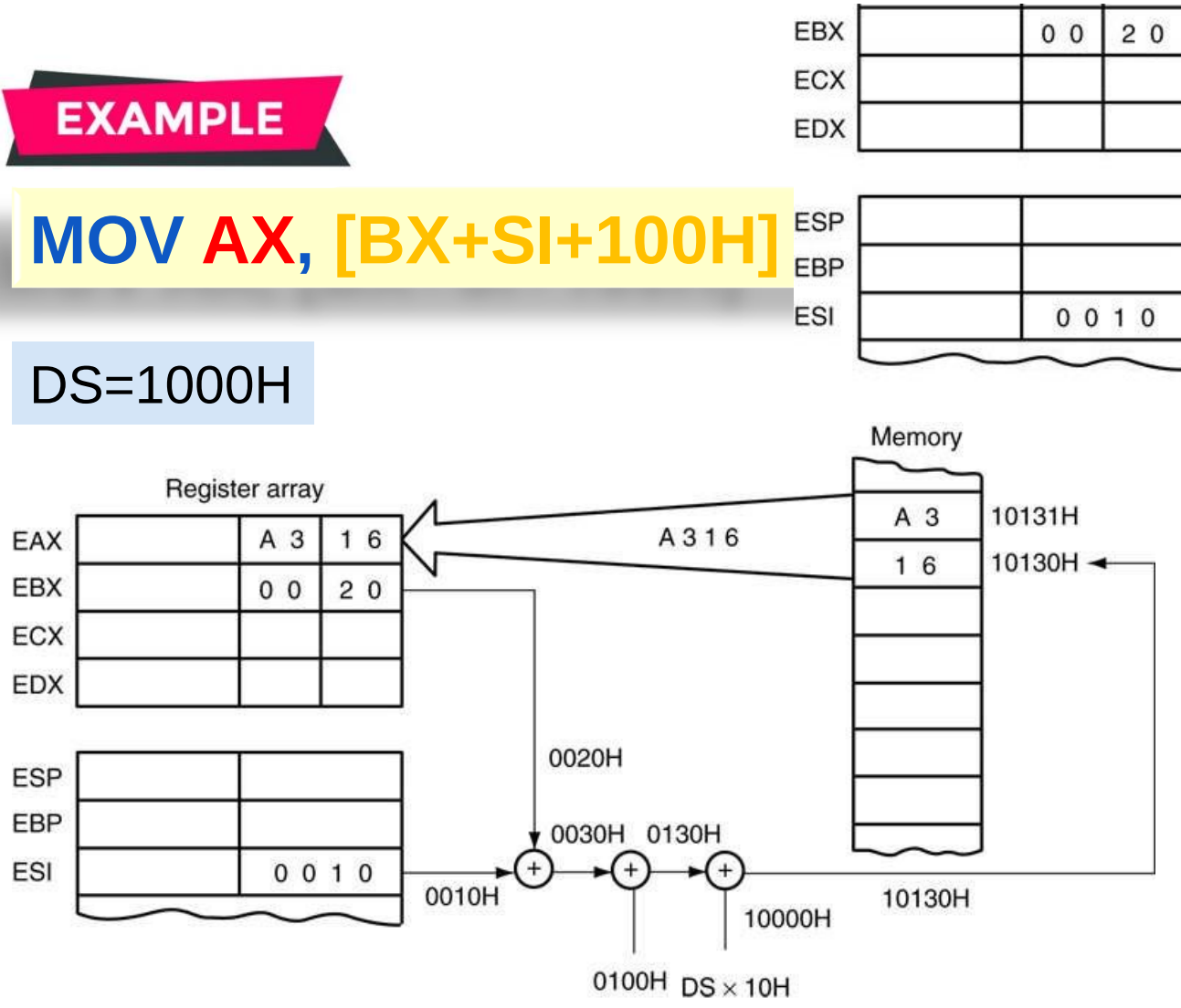
BASE RELATIVE-PLUS-INDEX ADDRESSING

- Least-used addressing mode.
- The following figure shows how data are referenced if the instruction executed by the microprocessor is MOV AX,[BX + SI + 100H].
  - displacement of 100H adds to BX and SI to form the offset address within the data segment.
- This addressing mode is too complex for frequent use in programming.

EXAMPLE

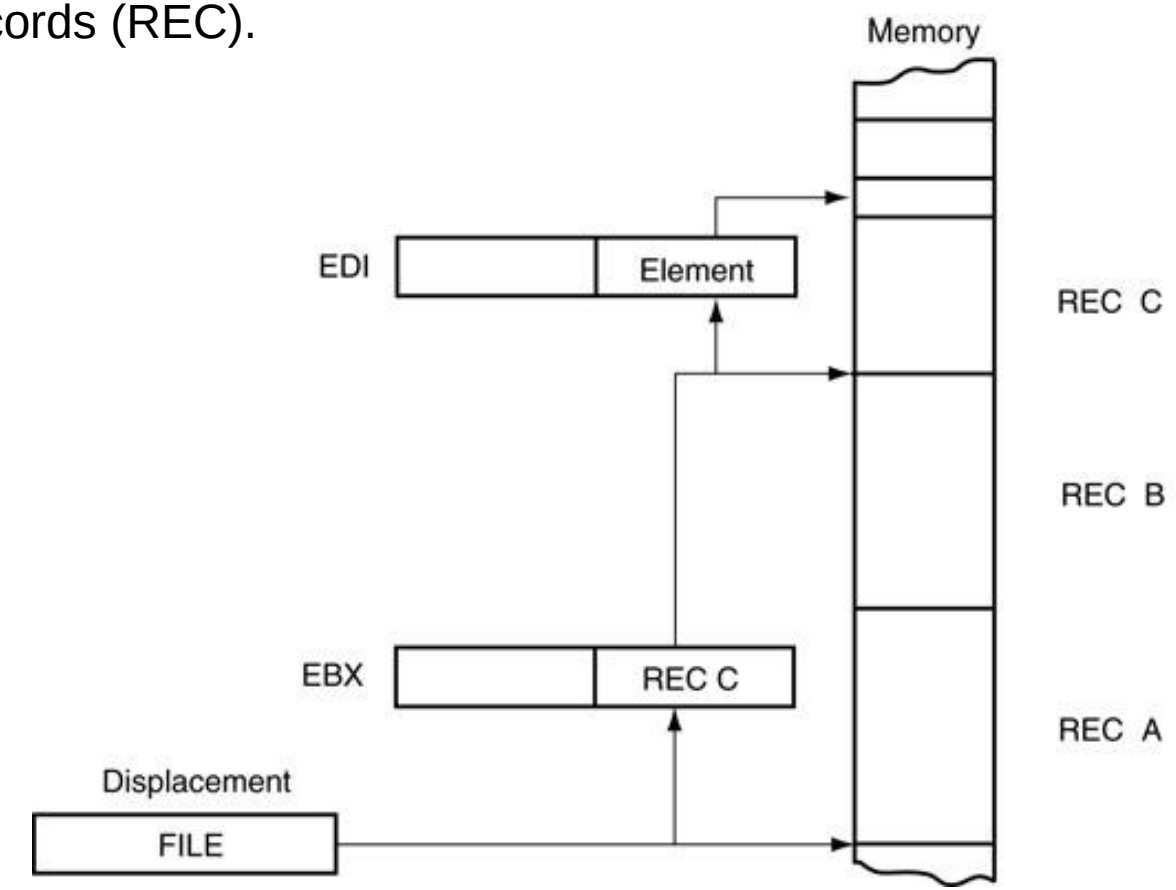
MOV AX, [BX+SI+100H]

DS=1000H



BASE RELATIVE-PLUS-INDEX ADDRESSING

- Base Relative Plus-Index addressing modes used to access a file that contains multiple records (REC).



SCALED INDEX ADDRESSING

- Unique to 80386 - Core2 microprocessors.
  - uses two 32-bit registers (a base register and an index register) to access the memory.
- The second register (index) is multiplied by a scaling factor.
  - the scaling factor can be 1x, 2x, 4x, 8x

Examples of scaled-index instructions.

MOV AX,[EBX + **2***ECX]

MOV [4 * ECX],EDX

MOV Instruction Addressing Modes

Type	Instruction	Source	Address Generation	Destination
Register	**MOV AX,BX**	Register BX	→	Register AX
Immediate	**MOV CH,3AH**	Data 3AH	→	Register CH
Direct	**MOV [1234H],AX**	Register AX	DS×10H + DISP = 10000H + 1234H →	Memory 11234H
Register Indirect	**MOV [BX],CL**	Register CL	DS×10H + BX = 10000H + 0300H →	Memory 10300H
Base-plus-index	**MOV [BX+SI],BP**	Register SP	DS×10H + BX + SI = 10000H + 0300H + 0200H →	Memory 10500H
Register Relative	**MOV CL,[BX+4]**	Memory 10304H	DS×10H + BX + 4=10000H+0300H+4 →	Register CL
Base Relative-plus-index	**MOV ARRAY[BX+SI],DX**	Register DX	DS×10H + ARRAY + BX + SI = 10000H + 1000H + 0300H + 0200H →	Memory 11500H
Scaled Index	**MOV [EBX+2×ESI],AX**	Register AX	DS×10H + EBX + 2×ESI = 10000H + 00000300H + 00000400H →	Memory 10700H

Notes: EBX = 00000300H, ESI = 00000200H, ARRAY = 1000H, DS = 1000H

REFERENCE TEXTBOOK

[1] “80X86 IBM PC and Compatible Computers: Assembly Language, Design, and Interfacing” by Muhammad Ali Mazidi , Janice Mazidi Gillispie and Janice Gillispie Mazidi

[2] “The Intel microprocessors 8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro processor, Pentium II, Pentium III, Pentium 4, and Core2 with 64-bit extensions: architecture, programming, and interfacing” by Barry B. Brey”

REFERENCES

[1] https://www.ee.hacettepe.edu.tr/~alkar/ELE336/w2-hacettepe[2016].pdf
[2] https://microcontrollerslab.com/8086-microprocessor-architecture/
[3] https://www.tutorialspoint.com/assembly_programming/assembly_registers.htm
[4] https://vlsifacts.com/category/dhd/verilog/